DevNexus | AgileBridge

SAS DOCUMENT

Prepared By

DevNexus

devnexus28@gmail.com

agilebridge
CONSIDERED SOFTWARE SOLUTIONS

Table of Contents

1.Introduction 2
Overview 2
Key Features 2
2. Architectural Requirements 3
2.1 Architectural Patterns 3
2.2 Design Patterns 7
2.3 Constraints 7
2.4 Architecture diagram 8
3. Mapping Quality Requirements to Architectural Decisions 9
4. Technology Requirements 15
5. API Contracts 18
6. Deployment Requirements: 39

1.Introduction

Overview

UniTrade is a peer-to-peer marketplace platform built for South African university students to buy and sell items such as textbooks, study materials and other students essentials. The platform verifies student identity before granting full access, uses AI-assisted risk scoring to screen listings before they go live and manages the entire transaction lifecycle, from browsing and reservation, through in-person meetups, to secure payment and handover confirmation. Payments are processed via Payfast and PIN-based system confirms that a handover has taken place. A reputation and dispute-handling system supports trust between users.

Key Features

- **Student verification** – Sign-up with university email domain checking, OTP verification and proof-of-registration upload, backed by an AI scanning layer with admin fallback for uncertain cases.
- **AI-assisted listing moderation** – Listings are automatically risk-scored (reserve-image checks, content classification) and routed to go live or held for admin review.
- **Marketplace browse** – View listings with title, price, condition, category and photos, reserve and add to your wishlist directly from the product card.
- **Reservations** – Express interest in an item to trigger an automated message and a time-boxed reservation, with clear acknowledgement and response windows.
- **In-app chat** – Coordinate directly with the other party to arrange meetup time and location through a meetup scheduler.
- **Secure payment** – Pay instantly via Payfast after inspecting the item in person.
- **PIN-based handover confirmation** – A PIN generated after payment confirms the item actually changed hands.
- **Ratings and reputation** – Rate the other party after a transaction, reputation scores factor into trust and abuse detection.
- **Dispute reporting** – flag listing-quality issues, no-shows, or false listing claims for admin review.

2. Architectural Requirements

Three concerns drive Unitrade's architecture. The platform has to act as a trusted intermediary trip between students who don't know each other, it has to work reliably

on mobile devices over limited connectivity, and it has to remain maintainable by a small team working across separate domains in parallel. The patterns described below follow from those concerns.

2.1 Architectural Patterns

Client-server (system wide).

The system is strictly client-server. A single thin client handles presentation and user interaction; all business logic, data storage, verification, payment confirmation, and moderation reside on the server. Clients cannot communicate directly with one another or bypass the server.

This is a deliberate choice rather than a default. UniTrade facilitates peer-to-peer exchange in the physical world — buyers and sellers meet in person — but the platform itself remains centralised, because verified identity, audit trails, and dispute resolution all require a trusted authority. A peer-to-peer architecture at the system level would undermine every trust guarantee the platform is built on. The accepted trade-off is that all transactional operations require server connectivity, there is no offline-first capability.

Layered (n-tier) architecture.

Five layers, each depending on the layer below it:

Layer	Responsibility
Presentation	Client UI and user input only
Edge	TLS termination, routing, and protection
API	Request validation, authorisation, routing — no business logic
Application	Core business logic, organised into domain modules
Data	Relational storage

The separation keeps business rules independent of transport concerns: changing a route does not require touching domain logic, and changing an external provider does not require touching a controller. The cost is additional indirection relative to placing logic directly in controllers, accepted because the alternative does not scale to multiple developers working concurrently.

Modular Monolith (Application layer).

All business domains are deployed as a single application but separated internally into fifteen modules: Identity, Listings, Listing Q&A, Reservations, Transactions, Chat, Disputes, Reputation, Reviews, Wishlist, Saved Searches, Notifications, Reference Data, Timetable, and Audit.

Microservices were considered and rejected. The domains are tightly coupled by nature: creating a reservation touches Listings, Reservations, Chat, Wishlist and Notifications in a single request, and distributing that operation would introduce distributed transactions, inter-service latency and operational overhead disproportionate to the team's size. The accepted trade-off is that modules cannot be scaled independently; the system scales as a whole. Module boundaries are defined by namespaces and module-owned interfaces and are maintained through code review (all modules share one assembly and one database context), so extracting a module later remains a refactor rather than a rewrite.

Python FastAPI AI Service (CLIP).

Although the core platform is a modular monolith, image classification is offloaded to a standalone, stateless Python FastAPI microservice. This service hosts an OpenAI CLIP PyTorch model (ViT-B/32) to compute 512-dimensional vector embeddings for multimodal risk scoring. It operates strictly within Azure Container Apps' internal VNet environment (non-public), and the .NET backend interacts with it via synchronous HTTP requests with a strict 5-second timeout. If the service is slow or unavailable the call fails open, and the listing is created without the image-similarity signal rather than blocking the user.

Advanced Algorithmic Solvers.

To handle computationally complex domain constraints, the application layer isolates specific tasks into dedicated algorithmic solvers executing in-memory:

- **Dynamic Programming Knapsack (BundleKnapsack):** Finds the optimal multi-seller textbook bundle under the buyer’s budget cap. The general knapsack problem is NP-hard, but Smart Budget maximises the number of items rather than an arbitrary value, so a dynamic program over item count, with each seller’s non-linear bundle discount folded into a per-count cost table, solves it exactly in $O(N^2)$ time.
- **1D Interval Sweepline Algebra (AvailabilityCalculator):** Computes mutual free time for physical meetups. It merges each student’s overlapping timetable commitments (O(B log B)), inverts them into free time within campus daylight hours (08:00–20:00), and intersects the two free lists with a two-pointer sweep (O(|A| + |B|)) to project conflict-free meetup windows.

Domain-Driven Design (module boundaries).

Each module constitutes a bounded context with its own models, repositories, and service interface and exception type. Modules interact through defined interfaces rather than reaching into one another's internal data. This is what makes the modular monolith genuinely modular rather than a folder convention. Some duplication of similar DTOs across boundaries is accepted deliberately as shared models are the primary mechanism by which bounded contexts erode.

State machine (Reservations module).

The reservation lifecycle is governed by centralised guarded transitions rather than status checks distributed across controllers, workers, and handlers. States progress from active to completed, expired, or cancelled, with each transition validated in one place and driven by either user action or system event. Keeping the lifecycle rules in a single location makes every transition predictable and auditable.

Communication patterns.

The system uses four distinct communication mechanisms selected per interaction type:

- **Request-response (primary) —** REST over HTTPS for browsing, registration, reservations, and payment initiation. The API is naturally resource-oriented, which maps cleanly onto REST semantics.

- **Push (real-time) —** WebSockets via SignalR for chat, reservation status changes, and payment events, avoiding the latency and load of client polling. SignalR does not queue messages for disconnected clients, so any flow depending on real-time delivery is paired with a fallback query.

- **Inbound webhooks (event-driven) —** the payment gateway notifies the server directly when a payment completes. This notification is a signature-verified and treated as the sole source of truth; the browser redirect that accompanies it is used only for user experience and confirms nothing, as it is client-controlled and therefore forgeable.

- **Background processing (asynchronous) —** background work that need not block a user request, such as reservation-expiry sweeps, no-show detection, proof-document retention, and suspension expiry, runs in dedicated in-process background workers (hosted services) on periodic timers.

Event-driven processing (asynchronous layer).

Each worker runs on its own periodic schedule and operates on its own slice of the data, which keeps user-facing request paths free of slow operations and allows new asynchronous behaviour to be added as an additional worker without modifying existing request logic.

Hosted Background Services (Workers).

To guarantee consistent system state without relying on incoming user requests, the architecture utilizes native ASP.NET Core BackgroundService timers operating independently of the main API request lifecycle. The following four critical workers execute on defined intervals:

- **ReservationExpiryWorker (60s):** Sweeps and terminates reservations that exceed the 24-hour mutual acceptance window.
- **NoShowDetectionWorker (60s):** Evaluates scheduled meetups against current time, automatically raising system-generated disputes for unfulfilled check-ins.
- **ProofRetentionWorker (24h):** Enforces statutory POPIA compliance by permanently purging unneeded student proof-of-registration binaries older than 30 days.
- **SuspensionExpiryWorker (15m):** Reinstates user accounts whose temporary administrative suspension periods have elapsed.

2.2 Design Patterns

Currently implemented:

- Repository - Every module exercises persistence through a repository interface rather than the data context directly, which isolates domain logic from Entity Framework and makes modules testable in isolation
- Observer - The real-time client maintains a subscriber collection for each event type, notifying all registered listeners when an event arrives. Server-side notifier interfaces follow the same shape
- Dependency Inversion - Modules depend on interfaces registered in the composition root, not concrete implementations which is what permits infrastructure to be substituted without changes to business logic

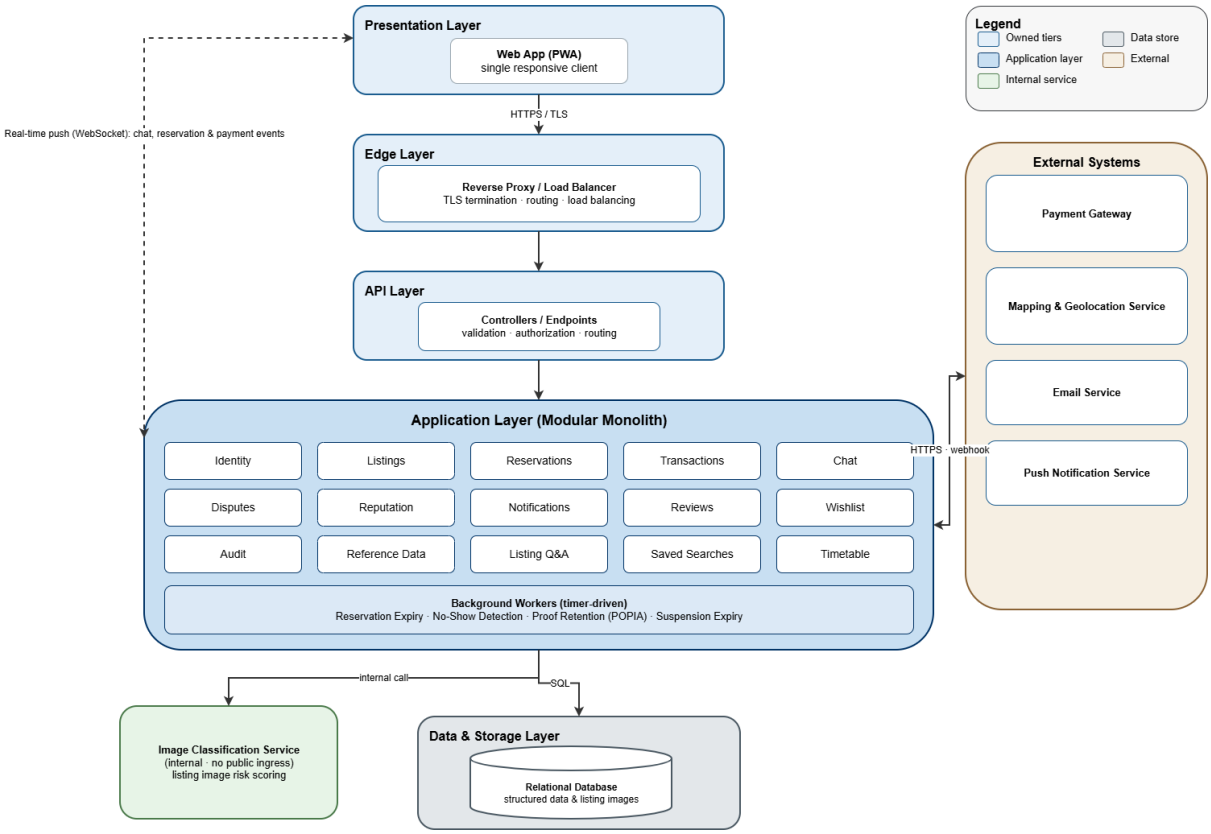
2.3 Constraints

1. **Payment gateway dependency.** The system uses PayFast as its sole payment gateway. The architecture must expose a publicly reachable webhook endpoint, treat redirect handling as separate from transaction verification, and store no card or banking credentials.
2. **University email validation.** Only students at recognised South African universities may register. Email domains are validated against the maintainable list and the domain determines institutional affiliation.
3. **Proof of registration accepted as PDF, JPG, or PNG (max 5 MB).** The document verification pipeline accepts PDF, JPG, and PNG uploads (up to 5 MB).
4. **AI is advisory only.** Automated scoring may approve, reduce visibility, or escalate, but may never issue a final rejection. Rejection remains a human decision.
5. **Administrative review SLA.** The system must maintain a visible, auditable moderation queue allowing flagged cases to be resolved within the defined service window.
6. **Optional location permissions.** Meetup verification may use location data, but users cannot be compelled to grant access. The architecture must support both verified and unverified check-ins.
7. **No refunds.** Payment states are forward-only. Buyer protection is provided by physical inspection prior to payment; disputes are handled reputationally rather than through transaction reversal.
8. **Mobile-first, PWA-based delivery.** The client is a single responsive Progressive Web App prioritising low-bandwidth operation and installability without app-store distribution.

9. **Data minimisation and POPIA compliance.** Only essential user and transaction data may be stored, and personal information must be protected in line with the Protection of Personal Information Act.
10. **Evolving external dependencies.** Constraints are subject to revision if gateway APIs, regulatory requirements, or the recognised institution lists change.

2.4 Architecture diagram

The diagram is organised into five architectural layers described above, together with an internal image-classification service and two further groupings: an asynchronous processing region containing the in-process background workers, and an external systems region containing the third-party dependencies, the payment gateway, mapping and geolocation services, and email and push-notification delivery.



3. Mapping Quality Requirements to Architectural Decisions

The SRS defines seven quality requirements. Each is mapped below to the architectural mechanisms intended to satisfy it.

Quality Requirement	Architectural Decision
Performance — Read endpoints respond at p95 ≤ 800 ms at 40 concurrent users	Composite and filtered database indexes on high-traffic read paths;
Scalability — At 2x baseline load, error rate <1% and p95 ≤ 1.5x baseline p95, with running instances increasing and no code change.	Stateless API layer behind the load balancer; horizontal auto-scaling on Azure Container Apps with defined min/max replica rules; database connection pooling; module boundaries that keep scaling to adding instances.
Security — 0 high/critical DAST findings (mediums triaged); 0 high/critical dependency CVEs; 100% of admin endpoints reject non-admin tokens; tampered/unsigned payment notification rejected with no state change; OTPs peppered and constant-time compared.	JWT authentication (token carried in an HttpOnly cookie) with role-based authorization enforced on all admin endpoints; server-side verification of the signed payment notification (ITN) before any state change; peppered + constant-time OTP-handling; TLS in transit and managed encryption at rest; secret and dependency scanning enforced in CI.

Reliability — Availability ≥ 99.5% over a rolling 30-day window; duplicate payment notification produces exactly one side effect; a single failed worker cycle does not stop the worker; a failed deployment auto-rolls-back.	Blue-green deployment in which a new revision is health-checked before receiving traffic and the prior revision remains live until the new one is verified, with automatic rollback on failure; idempotent webhook handling that tolerates duplicate delivery without duplicate side effects; per-cycle failure isolation in background workers so that a single failed cycle is logged without terminating the worker; managed database backups and platform-level health monitoring with automatic restart. Assumes a minimum of one always-on replica.
Maintainability — Static analysis runs on every merge (SonarCloud); formatting check passes; migrations apply cleanly from an empty database	Modular monolith with domain-aligned module boundaries, maintained through module-owned interfaces and code review, designed to confine most changes to a single module; automated CI enforcing build, unit and integration tests, formatting, and static analysis, and secret scanning before merge; automated deployment with database migrations applied as part of the release pipeline; separated unit and integration test suites, with integration tests executing against a real database engine rather than an in-memory substitute; centralised structured logging and monitoring.

Usability — 0 serious/critical accessibility violations (automated audit) across public and admin pages;	A single Progressive Web App serving web and mobile from one codebase, eliminating divergence between experiences; semantic, accessible markup using a shared design-token system;
Administration — Moderation queue returns cases oldest-first; every admin decision writes an audit entry with actor, timestamp and reason and dispatches a notification; dashboard surfaces any case open longer than the SLA of 48 h (verifications), 72 h (disputes).	A persisted verification/case entity with an explicit status lifecycle, providing a queryable moderation queue ordered by submission age; notification dispatch on every status-changing decision; audit logging of administrative actions with actor, timestamp, and reason retained for compliance.

ID	Quantified requirement	Tactic in SAS	Test / tool	Target / actual
QR-01	p95 read latency (feed, search, browse) at 40 concurrent users	Composite + filtered indexes on read paths	k6	≤ 800 ms, error < 1% / ≤ p95 42 ms, error 0% (40 VUs)
QR-02	Error rate + latency at 2x baseline load	Stateless API + Container Apps autoscale (min/max replicas) + DB connection pooling	k6 (baseline vs 2x)	p95(2x) ≤ 1.5x baseline, error < 1% / baseline p95 51 ms, peak p95 65 ms (<= 77 ms = 1.5x baseline); error 0% - PASS)
QR-03a	Admin endpoints reject non-admin callers	Role-based authorization on all admin routes	Integration test (non-admin token à 403, all routes)	100% reject / 100% reject

QR-03b	Forged/tampered payment callback has no effect	Server-side signed-ITN verification before state change	Integration + fault injection (tampered ITN)	Rejected, no state change /
QR-03c	OTP storage + comparison	Peppered hash + constant-time compare	Unit test	Pass / Pass
QR-03d	0 High, 0 Critical on frontend and backend staging.	Security headers (HSTS/CSP/anti-clickjacking), TLS-only, limited exposure	OWASP ZAP baseline	0 high,0 critical / 0 high,0 critical
QR-03e	No high/critical dependency CVEs on main	Dependency scanning enforced in CI	dotnet list package --vulnerable + npm audit	0 / 0
QR-04a	Duplicate payment notification	Idempotent ITN handling (dedup on provider reference)	Integration + duplicate delivery	Exactly one side effect / Exactly one side effect
QR-04b	Single failed worker cycle	Per-cycle failure isolation in background workers	Integration/Component + fault injection	Worker continues / Worker continues
QR-04c	Failed deployment	Blue-green + health-gated automatic rollback	Observed failed production deploy (automatic rollback) + pipeline config	Auto-rollback / Auto-rollback
QR-04d	Availability over a rolling 30-day window	Health checks + managed restart + min-replica (always warm)	UptimeRobot	≥ 99.5% / 100%

QR-05a	CI build & test gate: zero compile errors, 100% test pass rate, formatting enforced	Automated CI pipeline (Backend CI & Web Frontend CI) gating merges before staging deployment	GitHub Actions (dotnet build, format, xUnit; tsc, ESLint, Vitest) + SonarCloud	0 errors, 100% pass / 0 errors, 100% pass
QR-05b	Migrations apply cleanly from an empty database	Migrations run in the pipeline	Dotnet ef database update on a fresh Postgres (CI)	Applies clean / Applies clean
QR-06	Accessibility on public + admin pages	Semantic markup + tested design tokens	Lighthouse (axe-core) and axe-core Playwright checks	0 serious/critical / 0 serious/critical
QR-07a	Moderation queue returns cases oldest-first	Age-ordered queue query over a persisted case entity	Integration test	Oldest-first / Oldest-first
QR-07b	Every admin decision is audited	Audit log written on every status change (DB trigger for verification, audit service for disputes)	Integration test	actor + timestamp + reason present / actor + timestamp + reason present
QR-07c	Decision notifies the subject	Notification dispatched on each decision	Integration test (notification spy)	Dispatched / Dispatched
QR-07d	SLA breach is surfaced	Dashboard flags cases open longer than the window	Integration test	Case > 48 h (verifications) or > 72 h (disputes) flagged / Flagged

Perfomance and scalability testing environment (QR-01a, QR-02)

Load and scalability tests were executed against a local production configuration build (Docker, backend on :8080) rather than the deployed Azure staging environment. Azure Container Appscaling is constrained by subscription credit limits; the project runs on a shared limited tier where sustained high concurrency load testing would exhaust the available credits. The K6 scripts are environment agnostic and target URL injected via staging URL so the same test can be pointed at the deployment environment when higher capacity resources are available.

4. Technology Requirements

Frontend (Web): react + Vite (PWA):

We chose React as the foundation of Unitrade's client because it directly supports the project's core goals of scalability, usability, and efficient development. React's component-based architecture is well suited to building interactive, modular interfaces such as product listings, carts, messaging, and notifications, allowing features to be developed, tested, and updated independently. We build with **Vite** as the build tool for its fast development server and build times, and we ship the client as a **Progressive Web App** using **vite-plugin-pwa**, which gives us installability and push notifications on mobile devices without maintaining a separate native codebase.

Service worker architecture: the PWA uses a single shared service worker (firebase-messaging-sw.js), generated via injectManifest rather than the vite-plugin-pwa default of generateSW. This was a deliberate architectural choice as it lets us hand-write the Firebase Cloud Messaging (FCM) push-handling logic while still having vite-plugin-pwa inject the Workbox precache manifest into the same file instead of generating a second, competing service worker. The current service worker implements FCM push handling only; the injected manifest is not wired to runtime caching, so the app is installable but does not operate offline.

Backend API: ASP.NET Core(C#):

We use ASP.NET Core because it provides a secure, scalable, cloud-ready backend that fits naturally with our Azure hosting environment, simplifying deployment, scaling, and monitoring. Authentication and authorization use signed JWTs, validated by ASP.NET Core's JWT Bearer handler and carried in an HttpOnly cookie, giving us stateless, standards-based security with minimal custom code, which supports POPIA compliance. Overall, ASP.NET Core gives us reliable security, scalability, and efficient development for a growing student marketplace.

Primary Database: PostgreSql:

We use PostgreSQL as our primary database. Our data is highly structured and relational, requiring strong consistency and enforced relationships to meet POPIA requirements, and Postgres provides this along with encryption, backup, and scalability support. PostgreSQL also provides a strong foundation for AI-related capabilities we plan to build later, such as vector search, though this is future scope and not part of the current implementation.

Realtime Communication: SignalR

SignalR powers real-time communication across the application. This includes live chat between users, reservation updates, payment/PIN notifications, meetup updates, and other real-time user notifications, so users see changes as they happen rather than needing to refresh or poll for updates.

Mapping: Leaflet + OpenStreetMap + Nominatim:

We use Leaflet as our map rendering library, backed by OpenStreetMap map data and Nominatim for geocoding, to let users choose a location within a radius of where they currently are in order to propose a meet-up point for an exchange. This avoids the licensing and usage costs of a commercial mapping provider while giving us the location-picking functionality the marketplace needs.

Hosting and cloud infrastructure: Microsoft azure:

We host Unitrade on Microsoft Azure to provide a scalable, reliable, and low-maintenance platform. Azure Container Apps hosts and runs the application, pulling its container images from Azure Container Registry. The application stores its relational data in Azure Database for PostgreSQL, and performs asynchronous and background processing, such as reservation-expiry sweeps and background notification jobs, in dedicated in-process background workers (hosted services) that run on timers, keeping these tasks off the main request path. Transactional email is sent through Azure Communication Services, and real-time chat and notification updates are delivered to clients over WebSockets using ASP.NET Core SignalR, with Azure SignalR Service acting as the backplane so that real-time messages reach a user regardless of which backend

replica they are connected to. The managed ingress built into Azure Container Apps terminates TLS and load-balances across replicas, so the application does not use a separate CDN or edge-routing tier. Managed Identity lets Azure services authenticate to each other without storing credentials in code or configuration, Log Analytics provides centralised logging and diagnostics across these resources, and Resource Groups are provisioned per environment to keep environments isolated from one another.

CI/CD:

GitHub Actions automatically builds, tests, and deploys the application, with deployments to Azure Container Apps using a blue-green strategy. The full pipeline is covered in detail in the separate deployment document.

Payments: PayFast:

We use PayFast for payment processing. PayFast provides a fully featured developer sandbox with test credentials, letting us build and test the complete payment flow without waiting on business/merchant verification which is a good fit for a project still in active development. Payment confirmation is handled via PayFast's signed ITN (Instant Transaction Notification) webhook, rather than relying on the browser redirect after checkout, so a payment is only considered confirmed once we've received and verified a signed server-to-server notification from PayFast directly.

Notifications: FCM + Azure Communication Services

We use a multi-channel notification system, so students are reliably informed:

- **FireBase Cloud Messaging (FCM):** delivers instant push notification to the PWA
- **Azure Communication Services:** sends transactional emails (see Hosting and cloud infrastructure above).

Testing Stack:

- **Backend:** xUnit for test authoring, with Coverlet for code coverage collection.
- **Frontend:** Vitest for test authoring, with v8 for code coverage collection
- **End-to-end:** Playwright, exercising the application through a real or staged running instance.
- **Static analysis:** SonarCloud.

Secrets Management:

Local development uses local configuration files. Deployment secrets are stored in GitHub Encrypted Secrets and are injected into the deployment pipeline and application configuration at deployment time. Azure Key Vault is intentionally not part of the current architecture. We decided to do this as it was made on the basis that Key Vault would be overkill relative to the project's current scale and requirements.

Notes on Key Architectural Changes

The sections above describe our current technology stack. A few changes from earlier decisions are worth documenting, since they explain why the current architecture looks the way it does:

- **Resend → Azure Communication Services:** we moved away from Resend because, without a registered sending domain, emails could only be sent to our own verified addresses. Azure Communication Services allows us to send transactional emails to real users while fitting naturally into the rest of our Azure infrastructure.
- **MSSQL→ PostgreSQL:** PostgreSQL is a better fit for the project's future roadmap, particularly its support for AI-related capabilities such as vector extensions, while still providing the strong relational features the system requires today.
- **Ozow → PayFast:** Ozow requires a registered business before developer credentials are issued, which wasn't practical for development. PayFast provides a fully featured sandbox environment, better developer support, and let the team build and test the complete payment flow without waiting for business verification.

5. API Contracts

```
openapi: 3.0.3
info:
  title: UniTrade Admin & Disputes Api
  version: 1.0.0
servers:
  - url: /api/v1
security:
  - cookieAuth: []

paths:
```

```
  /admin/cases:
    get:
      summary: List pending cases,oldest first
      parameters:
        - {name: type, in: query, schema: {$ref:
'#/components/schemas/CaseType'}}
        - {name: status, in: query, schema: {type: string, default:
pending}}
        - {name: sort, in: query, schema: { type: string, default: age}}
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: {type: array, items: {$ref:
'#/components/schemas/CaseSummary'}}

  /admin/cases/{id}:
    get:
      summary: One case with its evidence
      parameters:
        - {name: id, in: path, required: true, schema: {type: string,
format: uuid}}
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: {$ref: '#/components/schemas/CaseDetail'}

  /admin/cases/{id}/decision:
```

```
  post:
    summary: Decide a case (also notifies the subject and writes
an audit entry)
    parameters:
     - {name: id, in: path, required: true, schema: {type: string,
format: uuid}}
    requestBody:
     required: true
     content:
      application/json:
       schema: { $ref: '#/components/schemas/DecisionRequest'}
    responses:
     '200':
      description: Updated case
      content:
       application/json:
        schema: {$ref: '#/components/schemas/CaseDetail'}

 /admin/cases/{id}/document:
  get:
    summary: Download a case's proof-of-registration document
    parameters:
     - {name: id, in: path,required: true, schema: {type:
string,format: uuid}}
    responses:
     '200':
      description: OK
      content:
       application/octet-stream:
        schema: {type: string, format: binary}
     '404':
```

```
        description: Not found

  /admin/users/{userId}/reputation:
    get:
      summary: Get a user's reputation summary, including strikes
      parameters:
        - {name: userId, in: path, required: true, schema: {type: string,
format: uuid}}
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: {$ref:
'#/components/schemas/UserReputationDto'}
        '404':
          description: User not found
          content:
            application/json:
              schema:
                type: object
                properties:
                  error: {type: string, example: user_not_found}

  /admin/users:
    get:
      summary: List users, filterable by verification
status/strikes/search term
      parameters:
        - {name: verificationStatus, in: query, schema: {type: string}}
        - {name: hasStrikes, in: query, schema: {type: boolean}}
```

```
      - {name: search, in: query, schema:{ type: string}}
      - {name: page, in: query, schema:{ type: integer, default: 1,
minimum: 1}}
      - {name: limit, in: query, schema:{ type: integer, default: 20,
minimum: 1, maximum: 100}}
    responses:
     '200':
      description: OK
      content:
       application/json:
        schema: {$ref:
'#/components/schemas/ListUsersResponseDto'}
 /admin/ping:
  get:
   summary: Health check
   security: []
   responses:
    '200':
     description: OK
     content:
      application/json:
      schema:
       type: object
       properties:
        ok: {type: boolean}

 /saved-searches:
  post:
   summary: Create a saved search
   requestBody:
    required: true
```

```
      content:
       application/json:
        schema: {$ref:
"#/components/schemas/CreateSavedSearchDto"}
    responses:
     '201':
      description: created
      content:
       application/json:
        schema: {$ref:
'#/components/schemas/SavedSearchDto'}
  get:
   summary: List the caller's saved searches
   responses:
    '200':
     description: OK
     content:
      application/json:
       schema: {type: array, items: {$ref:
'#/components/schemas/SavedSearchDto'}}

 /saved-searches/{id}:
  delete:
   summary: Delete a saved search owned by the caller
   parameters:
    - {name:  id, in: path,required: true, schema: {type: string,
format: uuid}}
   responses:
    '204':
     description:  Deleted
    '404':
```

```
        description: Not Found

  /saved-searches/{searchId}/listings:
    get:
      summary: get listings currently matching a saved search
      parameters:
       - {name: searchId, in: path, required: true, schema: {type:
string, format: uuid}}
      responses:
       '200':
        description: OK
        content:
         application/json:
          schema: {type: array, items: {$ref:
'#/components/schemas/UserListingDto'}}
       '404':
        description: Saved search not found or not owned by the
caller
        content:
         application/json:
          schema:
           type: object
           properties:
            error: {type: string}


  /admin/audit:
    get:
      summary: Audit entries
      parameters:
      - {name: entityId, in: query, schema: {type: string}}
```

```
    - {name: actorId, in: query, schema: {type: string, format: uuid}}
    responses:
      '200':
        description: OK
        content:
          application/json:
            schema: {type: array, items: {$ref:
'#/components/schemas/AuditEntry'}}

  /admin/users/{userId}/listings:
    get:
      summary: List a user's listings
      parameters:
      - {name: userId, in: query,required: true, schema: {type: string,
format: uuid}}
      - {name: limit, in: query, schema: {type: integer, default: 10,
minimum: 1, maximum: 50}}
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: {type: array, items: {$ref:
'#/components/schemas/UserListingDto'}}

  /disputes:
    post:
      summary: File a dispute (any type, including a user-raised no-
show)
      requestBody:
        required: true
```

```
      content:
       application/json:
        schema: { $ref: '#/components/schemas/FileDispute'}
    responses:
     '201':
      description: Case created
      content:
       application/json:
        schema:
         type: object
         properties:
          caseId: {type: string, format: uuid}

 /reservations/{reservationId}/snapshot:
  get:
   summary: The listing as it was when this reservation was made
   parameters:
    - { name: reservationId, in: path, required: true, schema:
{type: string, format: uuid}}
   responses:
    '200':
     description: OK
     content:
      application/json:
       schema: {$ref: '#/components/schemas/ListingSnapshot'}

 /listings/{id}/publish:
  post:
   summary: Publish a listing (existing endpoint; now requires a
verified seller)
   parameters:
```

```
      - {name: id, in: path, required: true, schema: {type: string,
format: uuid}}
    responses:
      '200': {description: Published}
      '403':
        description: seller is not verified
        content:
          application/json:
            schema:
              type: object
              properties:
                error: {type: string, example: SELLER_NOT_VERIFIED}

  /listings/{listingId}/questions:
    get:
      summary: List questions asked on a listing
      parameters:
        - {name: listingId, in: path, required: true, schema: {type:
string, format: uuid}}
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema: {type: array, items: {$ref:
'#/components/schemas/ListingQuestionDto'}}
    post:
      summary: Ask a question on a listing
      parameters:
        - {name: listingId, in: path, required: true, schema: {type:
string, format: uuid}}
```

```
    requestBody:
      required: true
      content:
        application/json:
          schema: {$ref: '#/components/schemas/AskQuestionDto'}
    responses:
      '200':
        description: Question created
        content:
          application/json:
            schema: {$ref:
'#/components/schemas/ListingQuestionDto'}
      '400':
        description: Blank question text
        content:
          application/json:
            schema: {$ref: '#/components/schemas/ErrorCode'}
      '403':
        description: Caller is the listing's own seller
      '404':
        description: Listing not found, or exists but not live.
        content:
          application/json:
            schema: {$ref: '#/components/schemas/ErrorCode'}


  /listings/questions/{questionId}/answer:
    post:
      summary: Answer a question (seller only).
      parameters:
```

```
      - {name: questionId, in: path, required: true, schema: {type:
string, format: uuid}}
    requestBody:
      required: true
      content:
        application/json:
          schema: {$ref:
'#/components/schemas/AnswerQuestionDto'}
    responses:
      '200':
        description: Question answered
        content:
          application/json:
            schema: {$ref:
'#/components/schemas/ListingQuestionDto'}
      '400':
        description: Already answered
        content:
          application/json:
            schema: {$ref:  '#/components/schemas/ErrorCode'}
      '403':
        description:  Caller is not the listing's seller
      '404':
        description: Question not found
        content:
          application/json:
            schema: {$ref: '#/components/schemas/ErrorCode'}



components:
```

```
securitySchemes:
  cookieAuth: {type: apiKey, in: cookie, name: authToken}
schemas:
  CaseType:
    type: string
    enum: [verification, listing_quality, report_listing, no_show]

  CaseDecision:
    type: string
    enum: [approve,reject, uphold,dismiss,request_info]

  Outcome:
    type: string
    enum: [strike, remove_listing, refusal_flag]

  CaseSummary:
    type: object
    properties:
      caseId: { type: string, format: uuid}
      type: {$ref: '#/components/schemas/CaseType'}
      status: {type: string}
      subjectUserId: {type: string, format: uuid}
      submittedAt: {type: string, format: date-time}
      ageHours: {type: number}
      slaHours: {type: integer}
      slaBreached: {type: boolean}
      title: {type: string, nullable: true}
      subjectInitials: {type: string, nullable: true}
      counterpartyInitials: {type: string, nullable: true}
      raisedBy: {type: string, format: uuid}
      buyerId: {type: string, format: uuid, nullable: true}
```

```
      sellerId: {type: string, format: uuid, nullable: true}
      reservationId: {type: string, format: uuid, nullable: true}
      subjectName: {type: string, nullable: true}
      subjectDegree: {type: string, nullable: true}
      hasDocument: {type: boolean}

    CaseDetail:
      allOf:
      - $ref: '#/components/schemas/CaseSummary'
      - type: object
        properties:
          subject: {$ref: '#/components/schemas/PartySummary'}
          counterParty: {$ref:
'#/components/schemas/PartySummary'}
          evidence: {$ref: '#/components/schemas/Evidence'}
          history: {type: array, items: {type: object}}
          filedByUserId: {type: string, format: uuid, nullable: true}
          filedByRole: {type: string}

    PartySummary:
      type: object
      properties:
        userId: {type: string, format: uuid}
        name: {type: string}
        initials: {type: string}
        faculty: {type: string, nullable: true}
        reviewAverage: {type: number}
        reputationScore: {type: number}
        strikeCount: {type: integer}

    CaseEvidence:
```

```
    type: object
    properties:
      university: {type: string, nullable: true}
      degree: {type: string, nullable: true}
      year: {type: integer, nullable: true}
      email: {type: string,nullable: true}
      domainValid: {type: boolean, nullable: true}
      snapshot: {$ref: '#/components/schemas/ListingSnapshot'}
      buyerPhotos: {type: array, items: {type: string}, nullable: true}
      sellerRefusedPhotos: {type: boolean, nullable: true}
      listingId: {type: string, format: uuid, nullable: true}
      reportReason: {type: string, nullable: true}

      meetupId: {type: integer, nullable: true, description: "int,not
uuid, same Meetup.MeetupId mismatch as elsewhere"}
      buyerCheckIn: {type: boolean, nullable: true}
      buyerCheckInTime: {type: string, format: date-time, nullable:
true}
      sellerCheckIn: {type: boolean, nullable: true}
      sellerCheckInTime: {type: string, format: date-time, nullable:
true}
      pinStatus: {type: string, nullable: true}
      checkInWindowClosesAt: {type: string, format: date-time,
nullable: true}

  VerificationEvidence:
    type: object
    properties:
      university: {type: string}
      degree: {type: string}
      year: {type: integer}
```

```
    email: {type: string}

  ListingQualityEvidence:
    type: object
    properties:
      snapshot: {$ref: '#/components/schemas/ListingSnapshot'}
      photos: {type: array, items: {type: string}}
      sellerRefusedPhotos: {type: boolean}

  ReportListingEvidence:
    type: object
    properties:
      listingId: {type: string, format: uuid}
      reportReason: {type: string }
      snapshot: {$ref: '#/components/schemas/ListingSnapshot'}

  NoShowEvidence:
    type: object
    properties:
      meetupId: { type: integer}
      buyerCheckedIn: { type: boolean}
      sellerCheckedIn: { type: boolean}
      pinStatus: {type: string, nullable: true}

  DecisionRequest:
    type: object
    required: [decision]
    properties:
      decision: {$ref: '#/components/schemas/CaseDecision'}
      outcome:
        type: array
```

```
        items: {$ref: '#/components/schemas/Outcome'}
      reason: {type: string}

  FileDispute:
    type: object
    required: [type]
    properties:
      type: {$ref: '#/components/schemas/CaseType'}
      reservationId:
        type: string
        format: uuid
        description: "Required for listing_quality and
no_show.No_show resolves its meetup via this field, not
meetupId."
      listingId: {type: string, format: uuid, description: "Required for
report_listing"}
      meetupId:
        type: string
        format: uuid
        nullable: true
        description: "Not used by the current filing implementation;
no_show disputes resolve the meetup via reservationId."
      photos:
        type: array
        items: {type: string}
        description: "Required for listing_quality unless
sellerRefusedPhotos is true"
      sellerRefusedPhotos: {type: boolean}
      description: {type : string, description: "Also carries the report
reason for report_listing"}
```

```
    ListingSnapshot:
      type: object
      properties:
        reservationId: {type: string, format: uuid}
        listingId: {type: string, format: uuid}
        title: {type: string}
        price: {type: number}
        condition: {type: string}
        photoRefs: {type: array, items: {type: string}}
        courseTags: {type: array, items: {type: string}}
        capturedAt: {type: string, format: date-time}
        description: {type: string}

    Strike:
      type: object
      properties:
        userId: {type: string, format: uuid}
        sourceCaseId: {type: string,format: uuid, nullable: true}
        type: {type: string}
        reason: {type: string}
        createdByAdminId: {type: string, format: uuid}
        createdAt: {type: string, format: date-time}
        strikeId: {type: string, format: uuid}

    AuditEntry:
      type: object
      properties:
        logId: {type: integer, format: int64, description:
"AuditLog.LogId is a bigint, not a uuid"}
        actorId: { type: string, format: uuid,nullable: true}
        action: {type: string}
```

```
    entityId: {type: string}
    entityType: {type: string}
    oldValue: {type: string, nullable: true}
    newValue: {type: string, nullable: true}
    reason: {type: string,nullable: true}
    timestamp: {type: string, format: date-time}

  UserListingDto:
   type: object
   properties:
    listingId: {type: string, format: uuid}
    title:  {type: string}
    status: {type: string}
    price:  {type: number}
    createdAt:  {type: string, format: date-time}
    imageUrl: {type: string, nullable: true}

  UserReputationDto:
   type: object
   properties:
    userId: {type: string, format: uuid}
    name: {type: string}
    email: {type: string}
    universityName: {type: string}
    degree: {type: string}
    year: {type: integer}
    verificationStatus: {type: string}
    reviewAverage: {type: number}
    reputationScore: {type: number}
    reviewCount: {type: integer}
```

```
      strikes: {type: array, items: {$ref:
'#/components/schemas/StrikeDto'}}

    StrikeDto:
      type: object
      properties:
        strikeId: {type: string, format: uuid}
        type: {type: string}
        reason: {type: string}
        sourceCaseId: {type: string, format: uuid, nullable: true}
        createdByAdminId: {type: string, format: uuid}
        createdAt: {type: string, format: date-time}

    ListUsersResponseDto:
      type: object
      properties:
        users: {type: array, items: {$ref:
'#/components/schemas/UserListItemDto'}}
        total: {type: integer}

    UserListItemDto:
      type: object
      properties:
        userId: {type: string, format: uuid}
        name: {type: string}
        email: {type: string}
        degree: {type: string}
        year: {type: integer}
        verificationStatus: {type: string}
        reviewAverage: {type: number}
        reputationScore: {type: number}
```

```
    strikeCount: {type: integer}

  CreateSavedSearchDto:
    type: object
    required: [query]
    properties:
      query: {type: string}
      categoryId: {type: number, nullable: true}
      minPrice: {type: number, nullable: true}
      maxPrice: {type: number, nullable: true}
      courseId: {type: integer, nullable: true}

  SavedSearchDto:
    type: object
    properties:
      searchId: {type: string, format: uuid}
      query: {type: string}
      categoryId: {type: integer, nullable: true}
      minPrice: {type: number, nullable: true}
      maxPrice: {type: number, nullable: true}
      courseId: {type: integer, nullable: true}
      isActive: {type: boolean}

  ListingQuestionDto:
    type: object
    properties:
      questionId: {type: string, format: uuid}
      listingId: {type: string, format: uuid}
      questionText: {type: string}
      answerText: {type: string, nullable: true}
      askedAt: {type: string, format: date-time}
```

```
      answeredAt: {type: string, format: date-time, nullable: true}
      askedInitials: {type: string, description: "'?' if the asker's
identity could not be resolved"}

    AskQuestionDto:
      type: object
      required: [questionText]
      properties:
        questionText: {type: string}

    ErrorCode:
      type: object
      properties:
        code: {type: string}
```

6. Deployment Requirements:

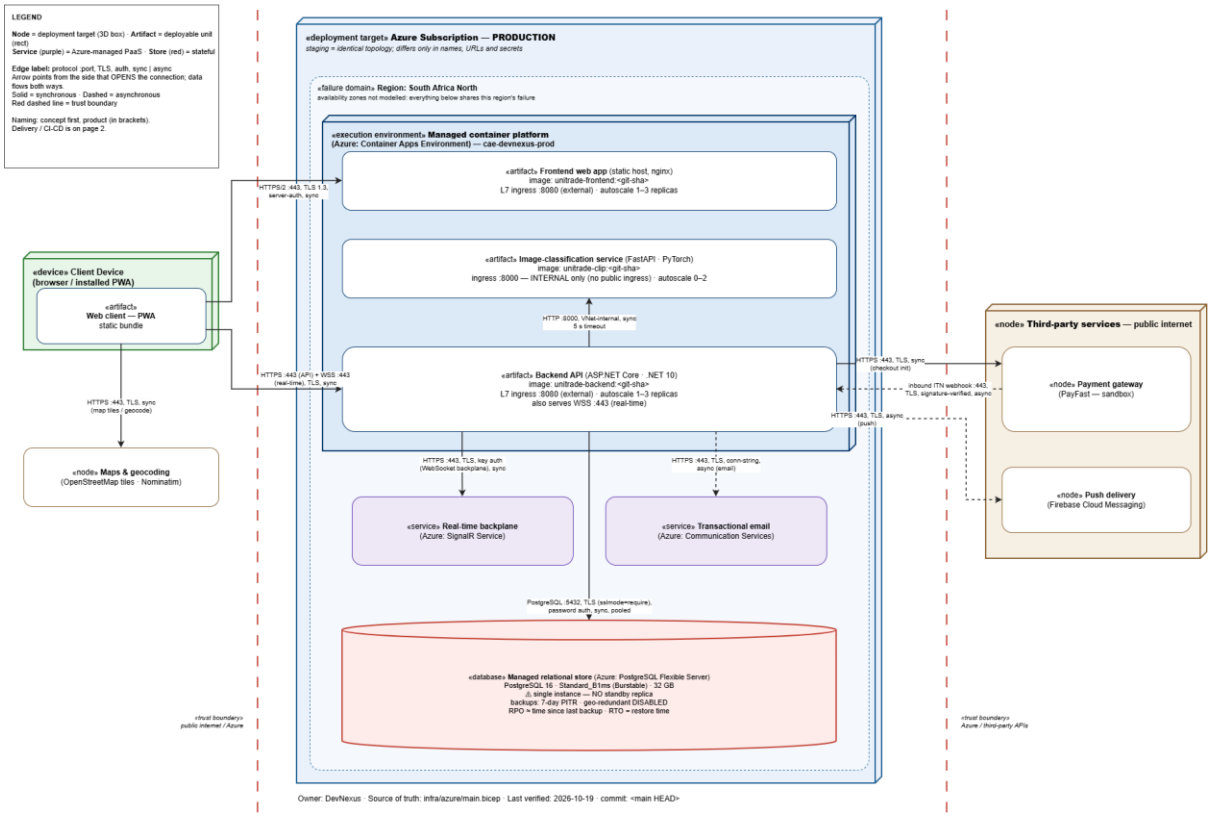
- **Live, Accessible System:**
  https://myunitrade.co.za
- **Environment Parity:** The system distinguishes between dev, staging, and production environments, each with its on azure container apps environment, PostgreSQL flexible Server, and resource group. All three share a single azure container registry. The main branch triggers an automated deployment to staging. Followed by a manual approval gate, that is approved by the devOps and team lead only, before promoting the same image to production.
- **Infrastructure as Code (IaC):**
  The system uses Azure Bicep templates to declaratively define and deploy all infrastructure: the Container Apps environment (with its Log Analytics workspace and Application Insights resource), the backend, frontend, and image-classification (CLIP) Container Apps, Azure Container Registry, PostgreSQL Flexible Server, and Communication Services. Deployments are

executed via `az deployment sub create` against modular Bicep templates (main.bicep+per source modules), parameterized per environment.

- **Secrets Management:**
  Credentials, connection strings, and API keys are never committed to the repository. All our secrets (ACR credentials, Postgres connection strings, JWT/otp signing secrets, Firebase config, Azure Communication Services connection string, and Azure OIDC identity values) are stored as encrypted Github Actions secrets and injected at deploy time via `az containerapp secret set` and `--set-env-vars`, referenced in the container app as `secretref:` values rather than plaintext. Azure authentication for the ci/cd pipeline uses openID connect (OIDC) via a user-assigned managed identity with federated credentials scoped to specific Github environments (staging, production), no long -lived azure credentials are stored in github.
- **Rollback Strategy:**
  Blue-green is used as our rollback strategy. Each deploy creates a new “green” container revision alongside the existing “blue” one. Traffic is shifted to green only after a health check succeeds. If deployment or the health check fails, the pipeline automatically shifts traffic back to the last know-good revision and deactivates the failed one.

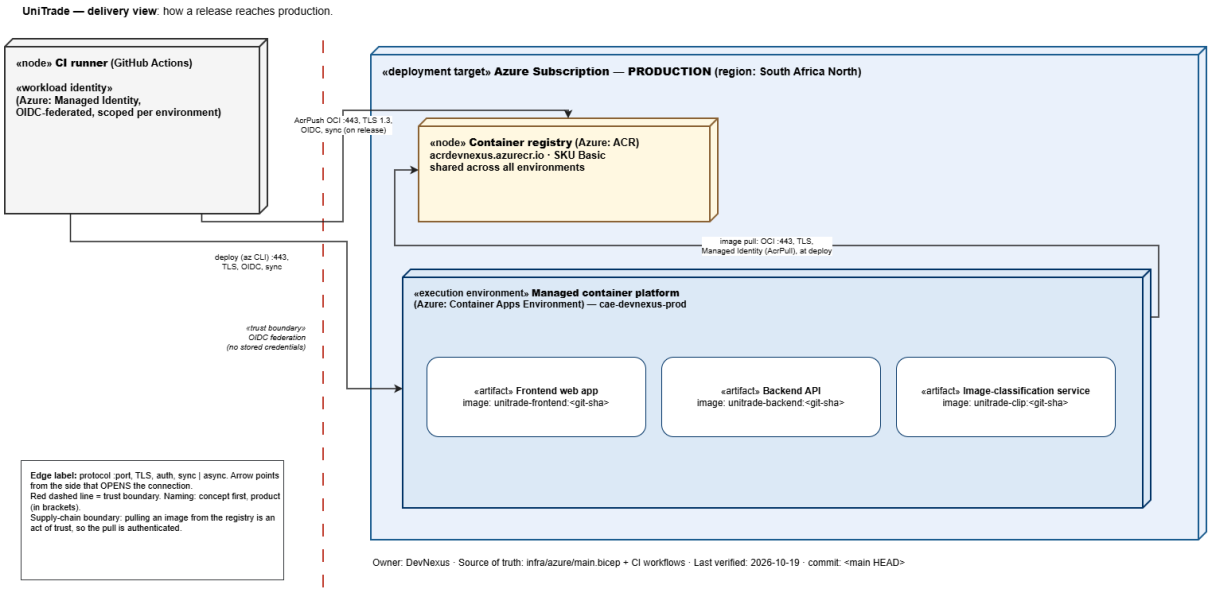
6.1 Deployment Diagram — Runtime View

The runtime view shows how the production system (region: South Africa North) is wired. The client — an installed PWA or browser — reaches only the frontend and backend over HTTPS and secure WebSockets; the backend calls the internal image-classification service over the private network, persists to a single Azure Database for PostgreSQL instance, and uses Azure SignalR for real-time messaging and Azure Communication Services for email. Edge labels give each connection's protocol, port, TLS, authentication and sync/async nature; the red dashed lines are the trust boundaries to the public internet and the third-party services (PayFast, Firebase, OpenStreetMap / Nominatim); and the relational store is drawn in red to flag it as the only stateful component and the system's single point of failure (single instance, no standby replica, 7-day point-in-time restore).


LEGEND
Node = deployment target (3D box) · Artifact = deployable unit (rect)
Service (purple) = Azure-managed PaaS · Store (red) = stateful
Edge label: protocol :port, TLS, auth, sync | async
Arrow points from the side that OPENS the connection; data flows both ways.
Solid = synchronous · Dashed = asynchronous
Red dashed line = trust boundary
Naming: concept first, product (in brackets).
Delivery / CI-CD is on page 2.
«deployment target» Azure Subscription — PRODUCTION
staging = identical topology; differs only in names, URLs and secrets
«failure domain» Region: South Africa North
availability zones not modelled: everything below shares this region's failure
«execution environment» Managed container platform
(Azure: Container Apps Environment) — cae-devnexus-prod
«artifact» Frontend web app (static host, nginx)
image: unitrade-frontend:<git-sha>
L7 ingress :8080 (external) · autoscale 1–3 replicas
«artifact» Image-classification service (FastAPI · PyTorch)
image: unitrade-clip:<git-sha>
ingress :8000 — INTERNAL only (no public ingress) · autoscale 0–2
HTTP :8000, VNet-internal, sync
5 s timeout
«artifact» Backend API (ASP.NET Core · .NET 10)
image: unitrade-backend:<git-sha>
L7 ingress :8080 (external) · autoscale 1–3 replicas
also serves WSS :443 (real-time)
«service» Real-time backplane
(Azure: SignalR Service)
«service» Transactional email
(Azure: Communication Services)
«database» Managed relational store (Azure: PostgreSQL Flexible Server)
PostgreSQL 16 · Standard_B1ms (Burstable) · 32 GB
⚠ single instance — NO standby replica
backups: 7-day PITR · geo-redundant DISABLED
RPO = time since last backup · RTO = restore time
«device» Client Device
(browser / installed PWA)
«artifact»
Web client — PWA
static bundle
«node» Maps & geocoding
(OpenStreetMap tiles · Nominatim)
«node» Third-party services — public internet
«node» Payment gateway
(PayFast — sandbox)
«node» Push delivery
(Firebase Cloud Messaging)
HTTPS/2 :443, TLS 1.3,
server-auth, sync
HTTPS :443 (API) + WSS :443
(real-time), TLS, sync
HTTPS :443, TLS, sync
(map tiles / geocode)
HTTPS :443, TLS, sync
(checkout init)
inbound ITN webhook :443,
TLS, signature-verified, async
HTTPS :443, TLS, async
(push)
HTTPS :443, TLS, key auth
(WebSocket backplane), sync
HTTPS :443, TLS, conn-string,
async (email)
PostgreSQL :5432, TLS (sslmode=require),
password auth, sync, pooled
«trust boundary»
public internet / Azure
«trust boundary»
Azure / third-party APIs
Owner: DevNexus · Source of truth: infra/azure/main.bicep · Last verified: 2026-10-19 · commit: <main HEAD>

6.2 Deployment Diagram — Delivery View

The delivery view shows how a release reaches production. The GitHub Actions runner authenticates to Azure through OIDC-federated workload identity — no credentials are stored — pushes the built container images to Azure Container Registry, and triggers the deployment via the az CLI; the Container Apps environment then pulls those images using its own managed identity (AcrPull). The red dashed line is the trust boundary crossed by OIDC federation, and the authenticated image pull marks the supply-chain boundary — pulling an image is an act of trust, so it is authenticated.



6.3 CI/CD Pipeline

The CI/CD pipeline (GitHub Actions) runs on every push and pull request. A gitleaks secret scan gates two reusable workflows — Backend CI and Web CI — which build, run unit and integration tests, enforce formatting and static analysis (SonarCloud), and scan dependencies and container images for high/critical CVEs (dotnet / npm audit and Trivy). Only after both pass does CD run, and only on pushes to main: it builds and pushes the backend, frontend and CLIP images to ACR, deploys to staging and — after a manual approval gate — to production, with each environment backing up the database, applying migrations, and cutting traffic over to a health-checked 'green' revision, rolling back traffic and migrations automatically on failure (blue-green). Separate workflows cover end-to-end tests (Playwright), Lighthouse audits, the NFR evidence report, and weekly Dependabot updates.

